Small Experimental Language
Introduction to language theory and compiling
Project – Part 1

Gilles Geeraerts Saina Sunny Arnaud Leponce

2026-2027

1 Introduction

Remark This project can be made by groups of maximum two students.

In this project, you are requested to design and write a compiler for, well, *Small Experimental Language*, aka Sel. The grammar of the language is given in Table 1 (page 2), where reserved keywords have been typeset using `typewriter font`. In addition, [VarName], and [Number] are lexical units, which are defined as follows (where the word *letter* is used for latin unaccented letter, i.e. A to Z). A [VarName] identifies a variable, which is a string of digits and letters, starting with a lowercase letter (this is case sensitive). A [Number] represents an integral numerical constant, and is made up of a string of digits only. The minus sign can be generated using rule [14].

For your information only (since the semantic is *not* important for the first part of the project), the @ operator is intended to print a variable on the standard output and `IN` to read an integer and store into the specified variable.

Finally, comments are allowed in Sel. There are two kinds of comments: Short comments, which are all the symbols appearing after a dollar sign `$` up to the end of the line, and long comments, which are all the symbols occurring between two number signs `##` keywords. There is no possibility of nested comments. Observe that comments do not occur in the rules of the grammar: they must be ignored by the scanner, and will not be transmitted to the parser.

Figure 1 shows an example of Sel program.

2 Assignment - Part 1

In this first part of the project, you must produce the *lexical analyzer* of your compiler, using the JFlex tool reviewed during the practicals.

Please adhere strictly to the instructions given below, otherwise we might be unable to grade your project, as automatic testing procedures will be applied.

2.1 Environment and provided files

The lexical analyzer will be implemented in JAVA. It must recognise the different lexical units of the language, and maintain a symbol table. To help you, several JAVA classes are provided on the UV:

[1]	<S>	→ `Main() { <Code> }`
[2]	<Code>	→ <Instruction> <Code>
[3]		→ ε
[4]	<Instruction>	→ <Assign> ;
[5]		→ <Loop>
[6]		→ <If>
[7]		→ `@` [VarName] ;
[8]		→ `IN` [VarName] ;
[9]	<Assign>	→ [VarName] `:=` <ExprArith>
[10]	<ExprArith>	→ [VarName]
[11]		→ [Number]
[12]		→ `(` <ExprArith> `)`
[13]		→ `-` <ExprArith>
[14]		→ <ExprArith> <Op> <ExprArith>
[15]	<Op>	→ `+`
[16]		→ `-`
[17]		→ `*`
[18]		→ `/`
[19]	<If>	→ `If (` <Cond> `) {` <Code> `}`
[20]	<Cond>	→ `0`
[21]		→ `1`
[22]		→ `(` <Cond> `)`
[23]		→ `!` <Cond>
[24]		→ <Cond> `||` <Cond>
[25]		→ <Cond> `&&` <Cond>
[26]		→ <ExprArith> <Comp> <ExprArith>
[27]	<Comp>	→ `==`
[28]		→ `<=`
[29]		→ `<`
[30]	<Loop>	→ `LOOP (`<Cond>`) {` <Code> `}`

Table 1: The SEL grammar.

```
## Euclid's algorithm ##

Main() {
   IN a;
   IN b;
   LOOP (0 < b) {
      c := b;
      LOOP (b <= a) { $ Computation of modulo
         a := a - b
      }
      b := a;
      a := c;
   }

   @a;
}
```

Figure 1: An example of SEL program.

- The `LexicalUnit` class contains an enumeration of all the possible lexical units;
- The `Symbol` class implements the notion of token. Each object of the class can be used to associate a value (a generic Java `Object`) to a `LexicalUnit`, and a line and column number (position in the file). The code should be self-explanatory. If not, do not hesitate to ask questions to the teacher, or to the teaching assistants.

2.2 Expected outcome

You must hand in all files required to compile and evaluate your code, as well as the proper documentation. This must be structured into five folders as follows, and a `Makefile` must be provided.

`src/` Contains all source files required to evaluate and compile your work:

- the JFlex source file `LexicalAnalyzer.flex` for your lexical analyzer;
- the generated file `LexicalAnalyzer.java`;
- provided files `Symbol.java` and `LexicalUnit.java` *without modification.*
- a `Main.java` file calling the lexical analyzer that reads the file given as argument and writes on the standard output stream the sequence of matched lexical units and the content of the symbol table (see below for details on the expected output).

`doc/` Contains the JAVADOC of your code, that will be used for reference in case we have trouble testing your project.

`test/` Contains all your example SEL files. It is necessary to provide relevant example files of your own.

`dist/` Contains an executable JAR called `part1.jar`. The command for running your executable should therefore be: `java -jar part1.jar sourceFile.sel`

`more/` Contains all other files.

`Makefile` At the root of your project (i.e. not in any of the aforementioned folders). There is an example Makefile on the UV (see the *Jflex practical* folder).

You will compress your root folder (in the *zip* format—no *rar* or other format), **which is named according to the following regexp**: `Part1_Surname1(_Surname2)?.zip`, where `Surname1` and, if you are in a group, `Surname2` are the last names of the student(s) (in alphabetical order); you are allowed to work in a group of maximum two students.

The *zip* file shall be submitted on Université Virtuelle before **October 21st, 2026, 23:59**, Brussels Grand-Place time.

2.3 Output of the program

The format of the output of the program must be:

1. First, the sequence of matched lexical units. You must use the *toString()* method of the provided Symbol class to print individual tokens;
2. Then, the word *Variables*, to clearly separate the symbol table from the sequence of tokens;

3. Finally, the content of the symbol table, formatted as the sequence of all recognized variables, in lexicographical (alphabetical) order. There must be one variable per line, together with the number of the line of the input file where this variable has been encountered for the first time (the variable and the line number must be separated by at least one space).

By default, your program must output on `stdout` (the terminal), and not write into a file.

For instance, on the following input:

```
Input(b)
```

your executable must produce exactly, using the `toString()` method of the `Symbol` class, the following output for the sequence of tokens (an example for the symbol table is given hereunder):

```
token: Input          lexical unit: INPUT
token: (              lexical unit: LPAREN
token: b              lexical unit: VARNAME
token: )              lexical unit: RPAREN
```

Note that the *token* is the matched input string (for instance `b` for the third token) while the *lexical unit* is the name of the matched element from the `LexicalUnit` enumeration (`VARNAME` for the third token).

Also, for the example in Figure 1, the symbol table must be displayed as:

```
Variables
a 4
b 5
c 7
```

An example input Sel file with the expected output is available on Université Virtuelle to test[1] your program.

3 Frequently Asked Questions

Here are some questions that we have gotten in the previous years which might help you during your project. If you have further questions please ask in the forum on the UV or via email.

- Q: What should happen if the Sel file can not be correctly tokenized?

  A: Throw a (useful) error message such as "Unknown symbol detected" or "Long comment not closed" for example. You can assume that you will not encounter a Sel file that has nested comments.

- Q: What about whitespaces?

  A: Whitespaces are not necessary beyond separating keywords, but they are nice for readability. All extra whitespaces must be ignored. For example:

[1] It is by no means extensive!

```
(If abbb Then  x=3:
```

```
( If abbb Then x=3 ;
```

```
(
    If      (    abbb
    )   {
        x := 3
    ;
```

must all yield the same matching[2] of lexical units, namely:

```
token: (            lexical unit: LPAREN
token: If           lexical unit: IF
token: (            lexical unit: RPAREN
token: abbb         lexical unit: VARNAME
token: )            lexical unit: LPAREN
token: {            lexical unit: LBRACK
token: x            lexical unit: VARNAME
token: :=           lexical unit: ASSIGN
token: 3            lexical unit: NUMBER
token: ;            lexical unit: SEMI
```

However, there are cases where spaces are required to isolate a keyword: `abbbLoopx` should be scanned as one variable name, and therefore, `(IF abbbLoopx=3` would not produce the same result as above.

- Q: What is the use of a `Makefile`?

  A: You need to provide a `Makefile` because your executable might not run on our machine, because of different JAVA versions (unavoidable because some use Windows, Linux, macOS). A working `Makefile` allows us to quickly produce an executable from your code. It takes a considerable amount of time if we need to figure out how to turn your code into an executable by hand. In addition, having a `Makefile` can streamline the testing and debugging of the code, so it is worthwhile to have for your sake as well.

[2]This is not syntactically correct SEL code, but the lexing should nonetheless work.